

# *HydraServe: Minimizing Cold Start Latency for Serverless LLM Serving in Public Clouds*

***Authors: Chiheng Lou, Sheng Qi, Chao Jin, Dapeng Nie, Haoran Yang, Yu Ding,  
Xuanzhe Liu, Xin Jin***

***School of Computer Science, Peking University ,Alibaba Group***



***Presented by Kaiqi Gu***





**Corresponding author:**  
**Xuanzhe Liu**

<http://www.liuxuanzhe.com>

## Projects:

- AI systems, cloud computing
- Distributed machine learning systems
- Systems software

## Papers:

- [ICLR 26] ROGA: Scaling Generalist Agents for Office Productivity Tasks via Tool Generation
- [TOSEM 25] SCOPE: Performance Testing for Serverless Computing
- [TheWebConf24] Towards Energy-efficient Federated Learning via INT8-based Training on Mobile DSPs
- [ACM Trans 23] FaaSLight: General Application-level Cold-start Latency Optimization for Function-as-a-Service in Serverless Computing



- ***Background***
- ***Design***
- ***Evaluation***
- ***Conclusion***



- ***Background***
- *Design*
- *Evaluation*
- *Conclusion*



- **The Era of "Million Models"**

- Enterprise Demand: Driven by unique business needs Custom LLMs.
- Ecosystem Scale: 1,000,000+ models on Hugging Face.
- Key Challenge: How to serve this massive "Long-tail" efficiently?

- **serverless LLM serving**

- the cloud platform automatically provisions GPUs for models based on the actual request load, and model owners are only charged for the resource consumption during request processing

# Background

- Th

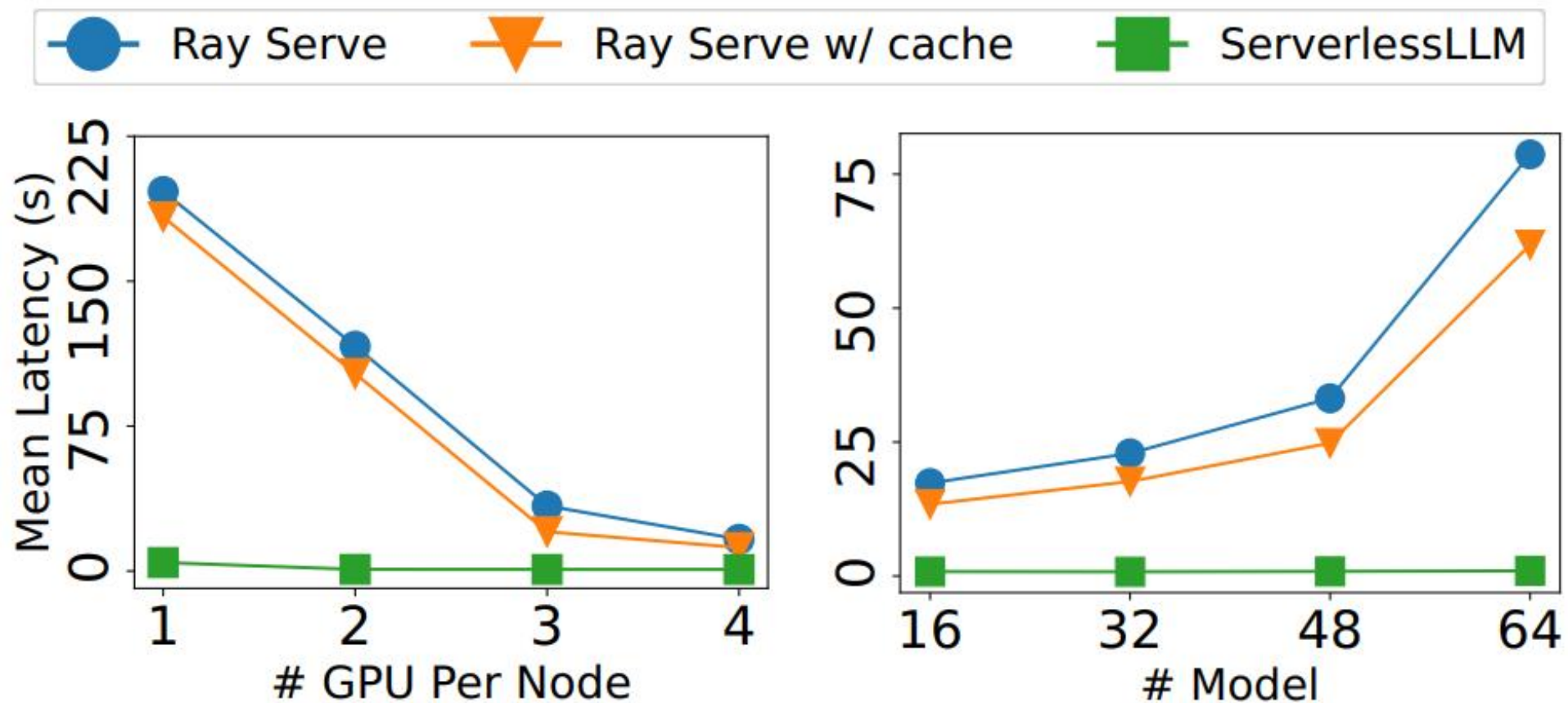
- 

- 

- 

- ser

- 



quest  
st



- **The Era of "Million Models"**

- Enterprise Demand: Driven by unique business needs Custom LLMs.
- Ecosystem Scale: 1,000,000+ models on Hugging Face.
- Key Challenge: How to serve this massive "Long-tail" efficiently?

- **serverless LLM serving**

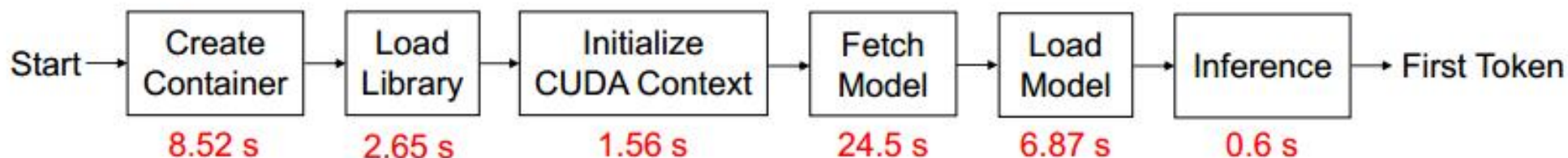
- the cloud platform automatically provisions GPUs for models based on the actual request load, and model owners are only charged for the resource consumption during request processing

**However, providing performance guarantees for serverless LLM users is challenging due to the inevitability of cold starts**

# Background

## Cold start latency breakdown

- **Container Creation.** The cluster controller allocates resources and creates a container on a GPU server.
- **Library Loading.** The container starts the Python runtime and imports libraries like PyTorch, along with LLM serving frameworks such as vLLM.
- **CUDA Context Initialization.** The runtime initializes the CUDA context to prepare for GPU tasks.
- **Model Fetching.** The serving framework retrieves the model from remote storage to local memory.
- **Model Loading.** The fetched model is loaded into memory and model states are initialized.
- **Inference.** Run the model and generate the first token.







- **Challenges**

- Large LLM sizes and **limited** public cloud network **bandwidth**.
- Cold-start stages run **sequentially** and incur high latency.
- Resource Contention among Colocated Containers.

- **Related work**

- caching LLMs in memory or SSDs
- retrieving model weights from active workers holding the same model

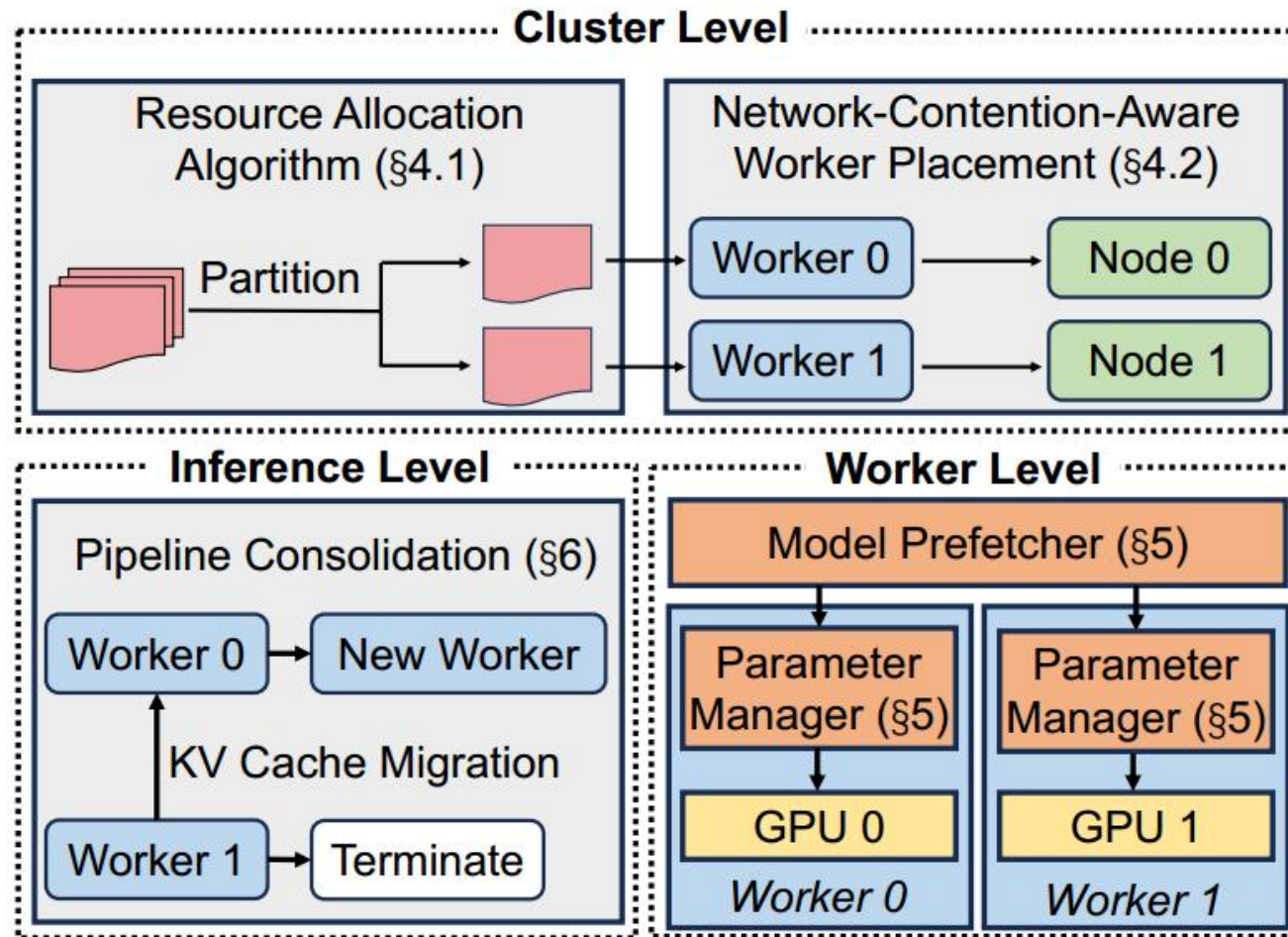
- **HydraServe work**

- Pipeline Parallelism
- Parallelizing the cold start process
- Smooth Migration and Integration



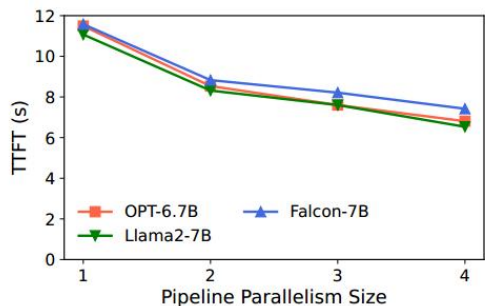
- *Background*
- ***Design***
- *Evaluation*
- *Conclusion*

- HydraServe system overview

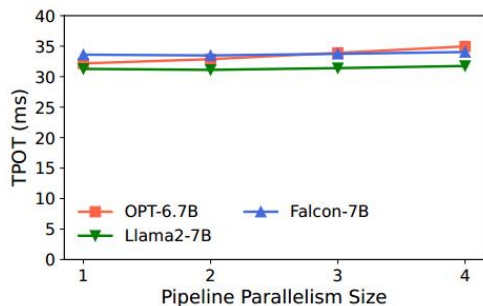


## • Cluster-Level Controller

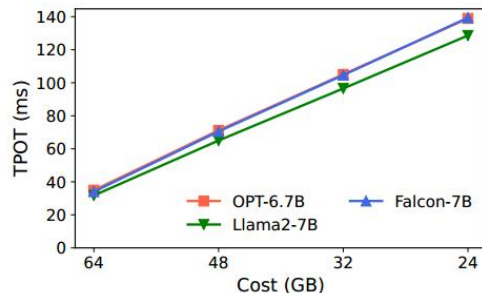
- **Resource Allocation.** Predict TTFT(Time To First Token) and TPOT(Time Per Output Token) under different configurations, and select the plan with minimal resource consumption that still satisfies performance requirements.
- **Network-Contention-Aware Worker Placement.** Strategically distribute cold-starting workers across different servers to ensure each instance obtains sufficient network bandwidth.



(a) TTFT of different pipeline parallelism sizes.



(b) Impact of pipeline parallelism size on TPOT.



(c) Impact of cost on TPOT.

### Algorithm 1 Resource Allocation Algorithm

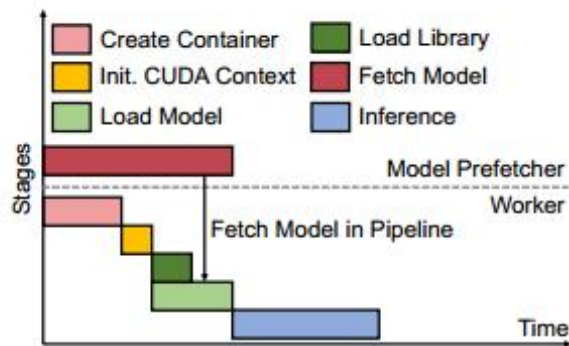
**Input:** time cost of container creation and runtime initialization  $t_c$ , data transmission  $t_n$ , prefill  $t_p$ , and decoding  $t_d$ ; model size  $M$ ; GPU server network bandwidth  $b_i$  and PCIe bandwidth  $p_i$ ; user specified requirements  $SLO_{TTFT}$  and  $SLO_{TPOT}$ .  
**Output:** pipeline parallelism size  $s$ , #full-memory workers  $w$ , and selected GPU servers  $g$ .

```

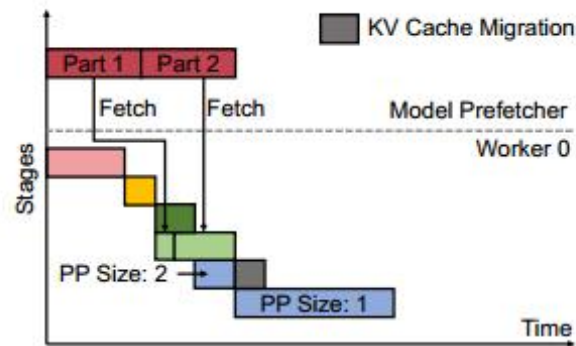
 $S \leftarrow \emptyset$ 
for  $s \in \{1, 2, \dots, 4\}$  do
    for  $w \in \{0, 1, \dots, s\}$  do
         $i_1, i_2, \dots, i_k \leftarrow$  Servers that fit a model of size  $M$ .
         $j_1, j_2, \dots, j_l \leftarrow$  Servers that fit a model of size  $M/s$ .
         $j'_1, \dots, j'_{l'} \leftarrow \text{MergeSort}((j_1, \dots, j_l), (i_{w+1}, \dots, i_k))$ 
         $g \leftarrow (i_1, i_2, \dots, i_w, j'_1, \dots, j'_{s-w})$ 
         $\text{max\_ratio} \leftarrow \max_{x \in g} \left( \frac{1}{b_x} + \frac{1}{p_x} \right)$ 
         $TTFT \leftarrow t_c + \frac{M}{s} \times \text{max\_ratio} + t_p \times (s - w + \frac{w}{s}) +$ 
             $t_n \times s$ 
         $TPOT \leftarrow t_d \times (s - w + \frac{w}{s}) + t_n \times s$ 
        if  $TTFT \leq SLO_{TTFT}$  and  $TPOT \leq SLO_{TPOT}$  then
             $S \leftarrow S \cup \{(s, w, g)\}$ 
if  $S$  is  $\emptyset$  then
    return  $(1, 1, (i_1))$   $\triangleright$  Use single worker if no solution
else
     $c \leftarrow$  Scheme that incurs minimal GPU sharing from  $S$ 
    return  $c$ 
    
```

## • Worker-Level Overlapping

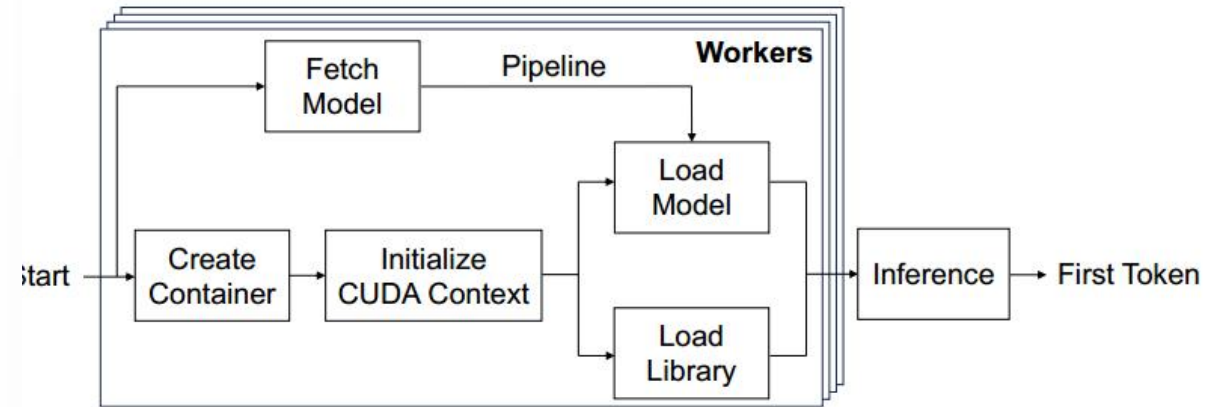
- **Model Prefetching.** By introducing a standalone prefetcher to pull models into shared memory during container creation, HydraServe parallelizes model downloading with runtime environment initialization.
- **Parameter Manager.** Within the container, the Parameter Manager uses multi-threading to overlap Python library loading with GPU parameter loading, further eliminating data transfer latency via zero-copy techniques.



(a) Prefetching whole model.



(b) Prefetching in two stages.



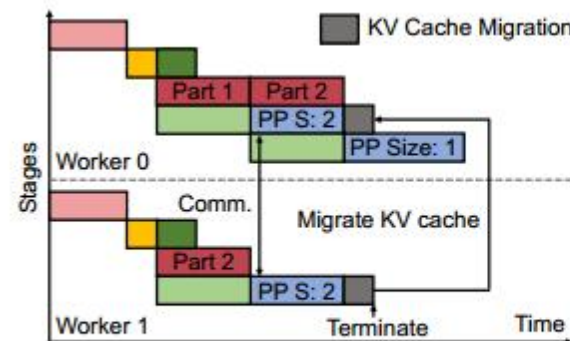


- **Inference-Level Consolidation**

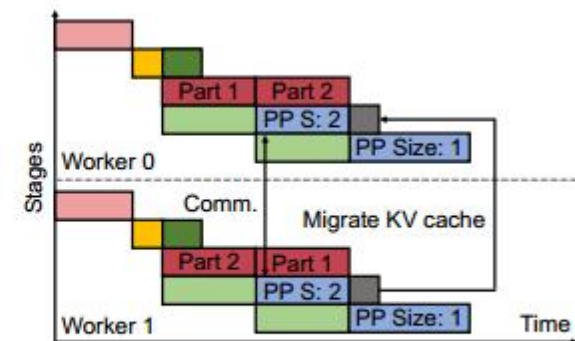
- **Worker Scaling.**

- *scaling down.* Consolidating parallel workers into one instance by migrating cache and releasing redundant resources
- *scaling up.* Transforming parallel workers into independent endpoints to handle sudden traffic spikes

- **Key-Value Cache Migration.** Consolidating distributed KV caches into the target node via streaming.



(c) Cold start with scaling down.



(d) Cold start with scaling up.



- *Background*
- *Design*
- ***Evaluation***
- *Conclusion*



- **Experimental Setup**

- **Testbed**

- I. Testbed 1 consists of 4 single A10 servers(188GB) and 4 four-card V100 servers(368GB) with 16 Gbps network bandwidth per node
- II. Testbed 2 includes 2 four-card A10 servers and 4 four-card V100 servers with 64 Gbps and 16 Gbps bandwidth respectively.

- **Baseline**

- I. ServerlessLLM: Low-latency serverless inference for large language models, in USENIX OSDI, 2024
- II. Serverless vLLM: Efficient memory management for large language model serving with pagedattention, in ACM SOSP, 2023.



# Evaluation

- Cold Start Latency

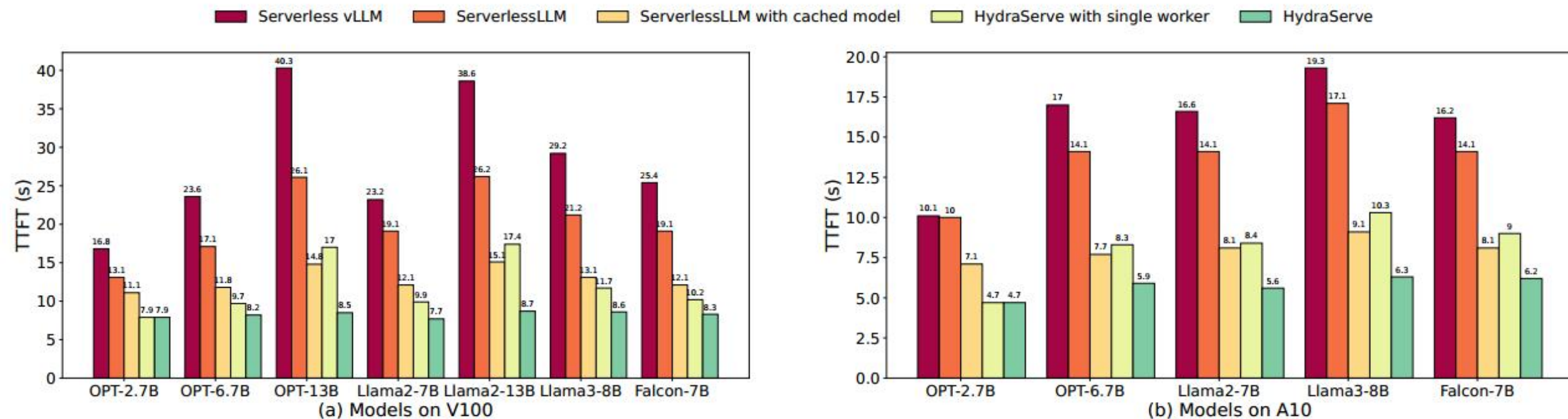


Figure 7: Cold start latency of systems for different models.

- TTFT SLO attainment of systems

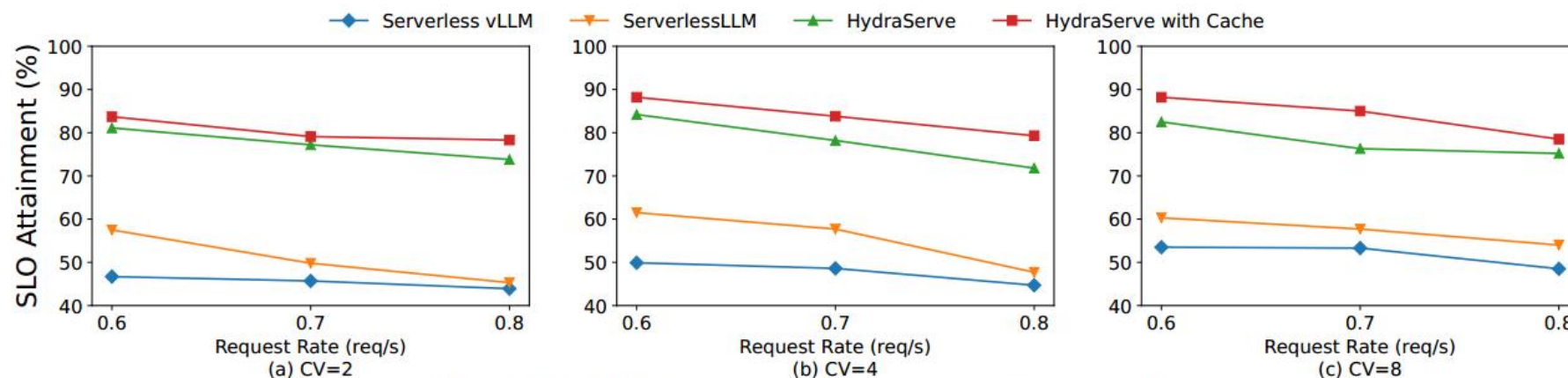


Figure 9: TTFT SLO attainment of systems under different CVs.

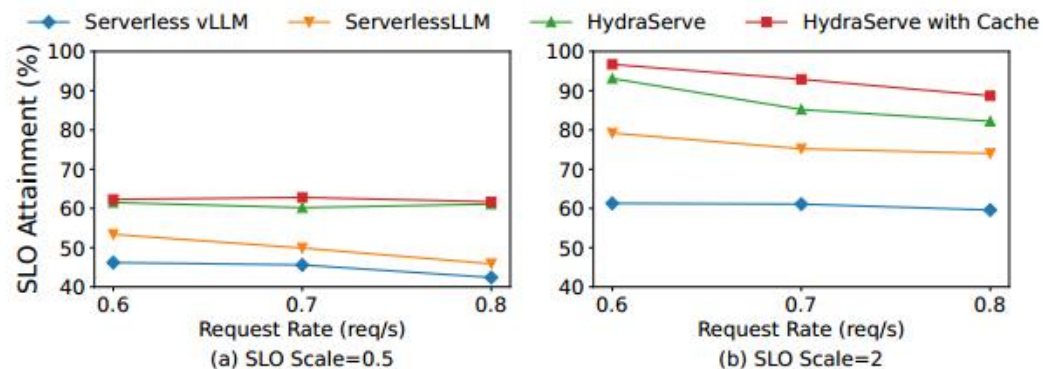
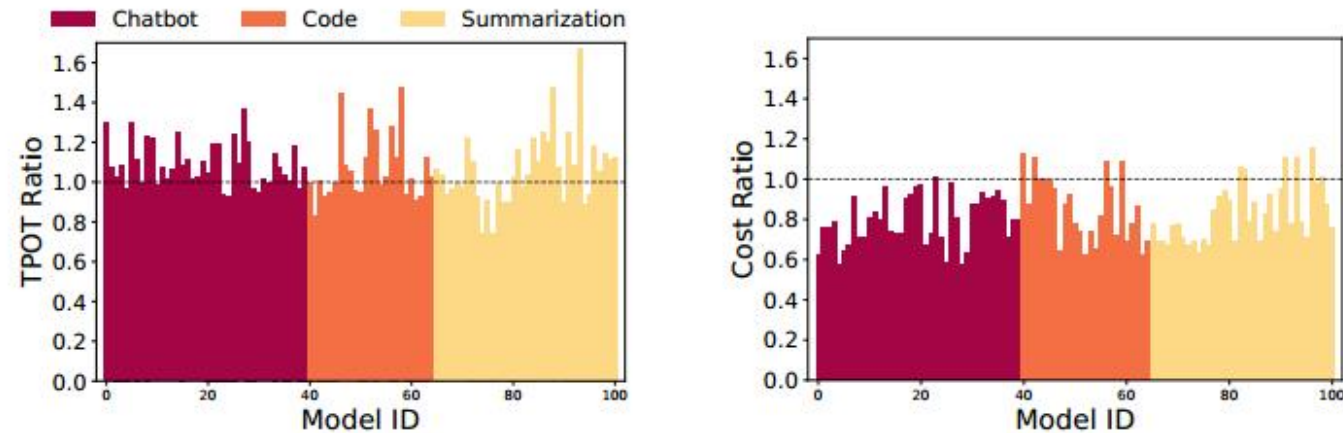


Figure 10: TTFT SLO attainment of systems under different SLOs.

- Trade-off of TPOT and cost



(a) TPOT ratios for different models. (b) Cost ratios for different models.

Figure 13: Relative TPOT and cost ratios (HydraServe vs. serverless vLLM) for different models.

# Evaluation

- Pipeline consolidation evaluation

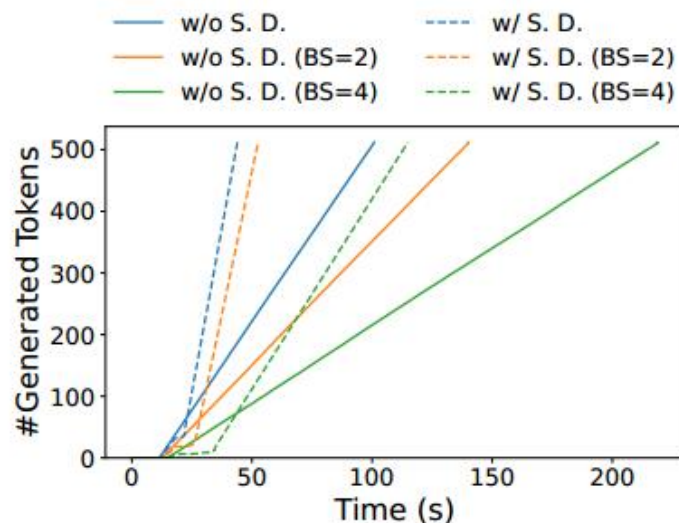
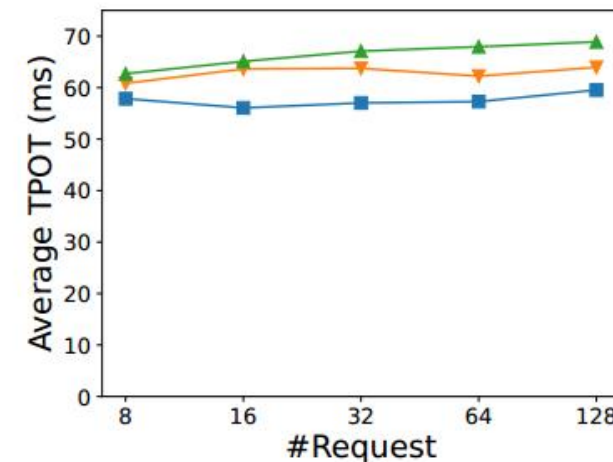
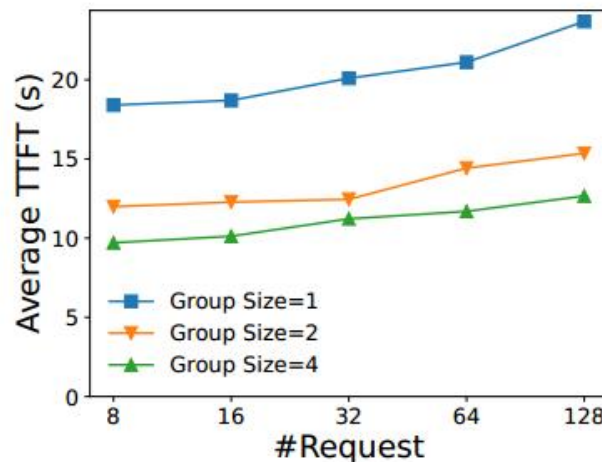


Figure 12: Total tokens generated over time for different inputs.



(a) Average TTFT of different loads. (b) Average TPOT of different loads.

Figure 14: Performance comparison for handling bursty loads with different parallel group sizes.

- **Brownfield evaluation**

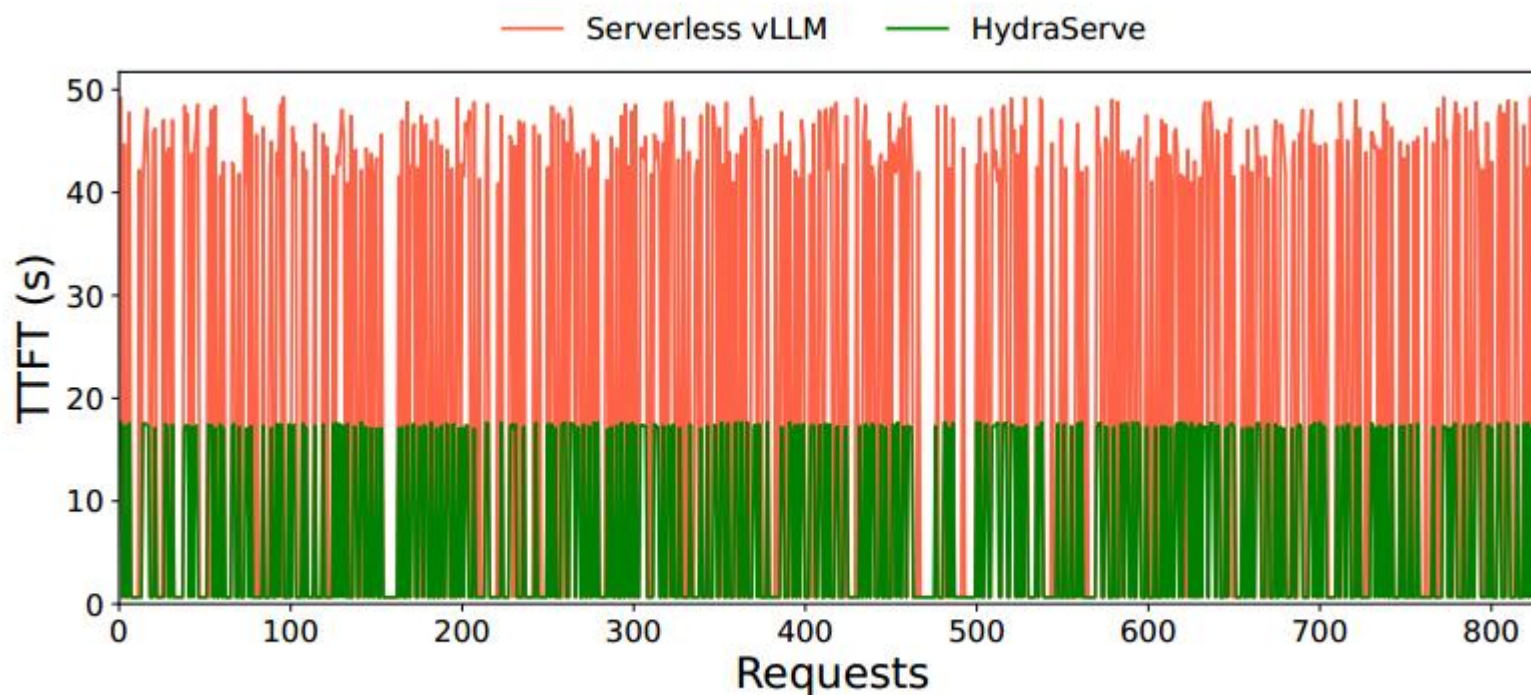


Figure 15: TTFT of requests in brownfield evaluation.



- *Background*
- *Design*
- *Evaluation*
- ***Conclusion***





- **Conclusion**

- HydraServe is a serverless LLM serving system that reduces cold-start latency in public clouds. It speeds up model fetching and runtime preparation via model distribution and stage overlapping, and uses pipeline consolidation for efficient resource usage.

- **Q1: Can it be applied to our scenario?**

- multi-tool calling involves multiple cold-started agent containers.

- **Q2: Can it be further improved?**

- Algorithm 1 Resource Allocation Algorithm can be improved.

- **Q3: Can it be generalized?**

- applicable to all services involving container creation and model fetching.